

University of Piraeus — Department of Digital Systems

Course: Cloud Computing and Advanced Application Architectures

Assignment: Development & Orchestration of a Microservices Application with Docker and Low■Code UI

Date: 2025-12-26

Project Theme: Word Game — API data selection & storage

Student(s): _____

1. Introduction & Scenario

This project implements a small microservices system for a “Word Game” theme. The user interacts with a Low■Code UI (Appsmith) to load data from two Mock REST APIs (word packs and powerups/hints) and save selected items into a local PostgreSQL database as “Favorites”. Additionally, the UI calls an external serverless API (dictionaryapi.dev) to fetch word definitions and stores logs in the local database.

Main use cases:

- Load “Wordpacks” from Mock API 1 and display them in the UI.
- Load “Powerups” from Mock API 2 and display them in the UI.
- Save a selected wordpack/powerup to PostgreSQL as a “Favorite”.
- Call an external serverless API to fetch definitions for a word and store the request/response as a log.

2. System Architecture, Technologies & Data Types

The system is orchestrated with Docker Compose and includes exactly five containers:

- UI: appsmith/appsmith-ce (Low■Code UI).
- Local Database: PostgreSQL (tables include favorites and api_logs).
- DB Admin: Adminer (web UI to inspect PostgreSQL).
- Mock REST API 1: svenwal/jsonplaceholder (seeded via a mounted db.json).
- Mock REST API 2: custom image built from a Dockerfile (json-server + db.json).

External serverless integration (not a container):

dictionaryapi.dev (public dictionary API, no API key required).

Key data objects:

- wordpacks: id, name, difficulty, description, words[]
- powerups: id, name, description, cost, effect
- favorites (PostgreSQL): player_name, source, item_type, item_id, item_name, item_payload (JSONB)
- api_logs (PostgreSQL): player_name, api_name, query, response_payload (JSONB), created_at

3. Services Implementation (Internal & External)

3.1 Appsmith UI (Low■Code)

The UI is implemented as an Appsmith application export (JSON). It includes widgets (Inputs, Buttons, Tables) and actions (REST API calls and PostgreSQL queries).

3.2 Mock APIs

- Mock API 1: svenwal/jsonplaceholder + mounted rest-apis/mock-api-1/db.json (endpoints: /wordpacks, /daily_challenges).
- Mock API 2: custom json-server image built from rest-apis/mock-api-2/Dockerfile + db.json (endpoints: /powerups, /hints, /achievements).

3.3 Local Database (PostgreSQL)

PostgreSQL is initialized using sql/init.sql (schema) and sql/export.sql (sample rows). The UI writes to favorites and api_logs to demonstrate storing user-selected API data and external API logs.

3.4 External Serverless API

The UI calls dictionaryapi.dev to fetch word definitions. The request and response are stored in api_logs for auditing and demonstration.

4. Docker Compose Orchestration & Installation

Prerequisites:

- Docker Engine / Docker Desktop
- Docker Compose plugin

Start:

```
docker compose -f compose.yaml up -d --build
```

Stop:

```
docker compose -f compose.yaml down
```

Main URLs:

- Appsmith UI: <http://localhost>
- Adminer: <http://localhost:8080>
- Mock API 1: <http://localhost:3000/wordpacks>
- Mock API 2: <http://localhost:3001/powerups>

5. Operation & Required Screenshots (to be added by the student)

Insert the following screenshots in the final submission. These are placeholders in this template PDF.

- Docker: output of “docker compose -f compose.yaml ps” showing 5 running containers.
- Appsmith dashboard: tables loaded with wordpacks and powerups.
- Appsmith: favorites table updated after saving a selected item.
- Appsmith: dictionary lookup results (definitions table).
- Adminer: favorites table rows showing stored selections.
- Adminer: api_logs table rows showing external API logs.

6. Bibliography / References

- Docker Compose documentation: <https://docs.docker.com/compose/>
- Appsmith documentation: <https://docs.appsmith.com/>
- json-server (typicode): <https://github.com/typicode/json-server>
- svenwal/jsonplaceholder image: <https://hub.docker.com/r/svenwal/jsonplaceholder>
- Adminer: <https://www.adminer.org/>
- dictionaryapi.dev: <https://dictionaryapi.dev/>